ASSIGNMENT 8: MEMORY MANAGEMENT IN xv6-RISC-V

Question 1 - Virtual-to-Physical Address Translation

The fork comparison showed the parent and child using the same virtual address for a global variable but different physical addresses. This is consistent with ordinary xv6 fork behaviour, where user pages are copied into the child's address space.

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu CPUS=1
balloc: first 1212 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M
-smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img
,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmi
o-bus.0

xv6 kernel is booting

init: starting sh
$ vatest
Global variable: VA=0x0000000000001000, PA=0x0000000087F1F000
Local variable: VA=0x0000000000003FAC, PA=0x0000000087F1DFAC
Heap variable: VA=0x0000000000013000, PA=0x0000000087F30000
Invalid address result: -1
Parent: VA=0x0000000000001000, PA=0x0000000087F1F000
Child:  VA=0x0000000000001000, PA=0x0000000087F48000
$
```

Question 2 - Physical Memory Statistics

A free_pages counter was added to the allocator state in kernel/kalloc.c alongside the free list and kmem.lock. kfree() increments the count when a page is returned; kalloc() decrements it when a page is successfully taken from the free list. Both changes happen while holding the allocator lock, protecting the counter from concurrent updates. get_free_pages() reads the count under the same lock.

Test stage	Free pages	Process pages	Interpretation
Initial	32530	5	Baseline process footprint
After allocating 10 pages	32520	15	Ten user pages consumed
Child after fork	32499	15	Fork temporarily used additional resources
Parent after child exits	32520	15	Child resources reclaimed after wait
After freeing allocation	32530	5	Free-page count returned to baseline

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu CPUS=1
o-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ memtest
meminfo struct size: 32

Initial memory
Total pages: 32731
Free pages: 32530
Process pages: 5
Page-table pages: 5
meminfo struct size: 32

After allocating 10 pages
Total pages: 32731
Free pages: 32520
Process pages: 15
Page-table pages: 5
meminfo struct size: 32

Child after fork
Total pages: 32731
Free pages: 32499
Process pages: 15
Page-table pages: 5
meminfo struct size: 32

Parent after child exits
Total pages: 32731
Free pages: 32520
Process pages: 15
Page-table pages: 5
meminfo struct size: 32

After freeing allocated memory
Total pages: 32731
Free pages: 32530
Process pages: 5
Page-table pages: 5
$ QEMU: Terminated
```

Question 3 - Page-table Dumping

The recursive virtual-address prefix is built from the page-table index at each level. Kernel diagnostics use printk(). The vmprinttest program prints the initial page table, calls sbrk(8 * 4096), shrinks by one page, and prints a third time. After heap growth, additional mappings appeared at virtual addresses from 0x5000 through 0xB000. After shrinking by one page, the mapping at 0xB000 disappeared while 0xA000 remained. This is consistent with the requested growth and shrink operation.

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu CPUS=1
xv6 kernel is booting

init: starting sh
$ vmprinttest

===== Initial page table =====
Page table dump:
  L2 index=0 table_pa=0x0000000087f21000 flags=1
  L1 index=0 table_pa=0x0000000087f20000 flags=1
  L0 index=0 VA=0x0000000000000000 PA=0x0000000087f22000 P=1 W=0 U=1
  L0 index=1 VA=0x0000000000001000 PA=0x0000000087f1f000 P=1 W=1 U=1
  L0 index=2 VA=0x0000000000002000 PA=0x0000000087f1e000 P=1 W=1 U=0
  L0 index=3 VA=0x0000000000003000 PA=0x0000000087f1d000 P=1 W=1 U=1
  L2 index=255 table_pa=0x0000000087f24000 flags=1
  L1 index=511 table_pa=0x0000000087f23000 flags=1
  L0 index=510 VA=0x0000003fffffe000 PA=0x0000000087f42000 P=1 W=1 U=0
  L0 index=511 VA=0x0000003ffffff000 PA=0x0000000080007000 P=1 W=0 U=0
Page-table dump complete.

===== After sbrk(8 pages) =====
Page table dump:
  L2 index=0 table_pa=0x0000000087f21000 flags=1
  L1 index=0 table_pa=0x0000000087f20000 flags=1
  L0 index=0 VA=0x0000000000000000 PA=0x0000000087f22000 P=1 W=0 U=1
  L0 index=1 VA=0x0000000000001000 PA=0x0000000087f1f000 P=1 W=1 U=1
  L0 index=2 VA=0x0000000000002000 PA=0x0000000087f1e000 P=1 W=1 U=0
  L0 index=3 VA=0x0000000000003000 PA=0x0000000087f1d000 P=1 W=1 U=1
  L0 index=4 VA=0x0000000000004000 PA=0x0000000087f26000 P=1 W=1 U=1
  L0 index=5 VA=0x0000000000005000 PA=0x0000000087f54000 P=1 W=1 U=1
  L0 index=6 VA=0x0000000000006000 PA=0x0000000087f53000 P=1 W=1 U=1
  L0 index=7 VA=0x0000000000007000 PA=0x0000000087f49000 P=1 W=1 U=1
  L0 index=8 VA=0x0000000000008000 PA=0x0000000087f46000 P=1 W=1 U=1
  L0 index=9 VA=0x0000000000009000 PA=0x0000000087f43000 P=1 W=1 U=1
  L0 index=10 VA=0x000000000000a000 PA=0x0000000087f27000 P=1 W=1 U=1
  L0 index=11 VA=0x000000000000b000 PA=0x0000000087f28000 P=1 W=1 U=1
  L2 index=255 table_pa=0x0000000087f24000 flags=1
  L1 index=511 table_pa=0x0000000087f23000 flags=1
  L0 index=510 VA=0x0000003fffffe000 PA=0x0000000087f42000 P=1 W=1 U=0
  L0 index=511 VA=0x0000003ffffff000 PA=0x0000000080007000 P=1 W=0 U=0
Page-table dump complete.
```

```
===== After sbrk(8 pages) =====
Page table dump:
  L2 index=0 table_pa=0x0000000087f21000 flags=1
  L1 index=0 table_pa=0x0000000087f20000 flags=1
  L0 index=0 VA=0x0000000000000000 PA=0x0000000087f22000 P=1 W=0 U=1
  L0 index=1 VA=0x0000000000001000 PA=0x0000000087f1f000 P=1 W=1 U=1
  L0 index=2 VA=0x0000000000002000 PA=0x0000000087f1e000 P=1 W=1 U=0
  L0 index=3 VA=0x0000000000003000 PA=0x0000000087f1d000 P=1 W=1 U=1
  L0 index=4 VA=0x0000000000004000 PA=0x0000000087f26000 P=1 W=1 U=1
  L0 index=5 VA=0x0000000000005000 PA=0x0000000087f54000 P=1 W=1 U=1
  L0 index=6 VA=0x0000000000006000 PA=0x0000000087f53000 P=1 W=1 U=1
  L0 index=7 VA=0x0000000000007000 PA=0x0000000087f49000 P=1 W=1 U=1
  L0 index=8 VA=0x0000000000008000 PA=0x0000000087f46000 P=1 W=1 U=1
  L0 index=9 VA=0x0000000000009000 PA=0x0000000087f43000 P=1 W=1 U=1
  L0 index=10 VA=0x000000000000a000 PA=0x0000000087f27000 P=1 W=1 U=1
  L0 index=11 VA=0x000000000000b000 PA=0x0000000087f28000 P=1 W=1 U=1
  L2 index=255 table_pa=0x0000000087f24000 flags=1
  L1 index=511 table_pa=0x0000000087f23000 flags=1
  L0 index=510 VA=0x0000003fffffe000 PA=0x0000000087f42000 P=1 W=1 U=0
  L0 index=511 VA=0x0000003ffffff000 PA=0x0000000080007000 P=1 W=0 U=0
Page-table dump complete.

===== After shrinking by 1 page =====
Page table dump:
  L2 index=0 table_pa=0x0000000087f21000 flags=1
  L1 index=0 table_pa=0x0000000087f20000 flags=1
  L0 index=0 VA=0x0000000000000000 PA=0x0000000087f22000 P=1 W=0 U=1
  L0 index=1 VA=0x0000000000001000 PA=0x0000000087f1f000 P=1 W=1 U=1
  L0 index=2 VA=0x0000000000002000 PA=0x0000000087f1e000 P=1 W=1 U=0
  L0 index=3 VA=0x0000000000003000 PA=0x0000000087f1d000 P=1 W=1 U=1
  L0 index=4 VA=0x0000000000004000 PA=0x0000000087f26000 P=1 W=1 U=1
  L0 index=5 VA=0x0000000000005000 PA=0x0000000087f54000 P=1 W=1 U=1
  L0 index=6 VA=0x0000000000006000 PA=0x0000000087f53000 P=1 W=1 U=1
  L0 index=7 VA=0x0000000000007000 PA=0x0000000087f49000 P=1 W=1 U=1
  L0 index=8 VA=0x0000000000008000 PA=0x0000000087f46000 P=1 W=1 U=1
  L0 index=9 VA=0x0000000000009000 PA=0x0000000087f43000 P=1 W=1 U=1
  L0 index=10 VA=0x000000000000a000 PA=0x0000000087f27000 P=1 W=1 U=1
  L2 index=255 table_pa=0x0000000087f24000 flags=1
  L1 index=511 table_pa=0x0000000087f23000 flags=1
  L0 index=510 VA=0x0000003fffffe000 PA=0x0000000087f42000 P=1 W=1 U=0
  L0 index=511 VA=0x0000003ffffff000 PA=0x0000000080007000 P=1 W=0 U=0
Page-table dump complete.
$ QEMU: Terminated
```

Question 4 - Lazy Memory Allocation

Lazy allocation separates virtual address reservation from physical page allocation. For positive sbrk(n), sys_sbrk() increases p->sz without calling growproc() to allocate every page. It returns the old size as the start of the reserved region. For negative n, uvmdealloc()/uvmunmap() shrinks the region and frees pages that were actually mapped, while skipping absent pages.

lazytest reserved 32 MB with sbrk(), measured memory, touched four widely spaced offsets, forked a child, and used a pipe read to write into an untouched buffer. The output reported “Touched pages: A B C D”, “Child: A D”, “Untouched read buffer: Z”, and “Lazy allocation test completed”.

```
shaurya@Shaurya6480:~/xv6-riscv$ make qemu

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ lazytest
Before sbrk: free pages=32529, process pages=5
After sbrk: free pages=32529, process pages=8197
Touched pages: A B C D
After touching pages: free pages=32522, process pages=8197
Child: A D
Untouched read buffer: Z
After fork and read: free pages=32520, process pages=8197
Lazy allocation test completed
$
```